```
"""Alert generation and threshold-based alerting rules.

This module defines the :class:`Alert` data type, representing a single
notable event raised about a sensor, and :class:`AlertRule`, which inspects
:class:`~src.readings.SensorReading` instances against a configured
acceptable range and produces an :class:`Alert` when a reading falls
outside of it.
"""

from __future__ import annotations

from datetime import datetime
from typing import Optional

from src.readings import SensorReading


class Alert:
    """A notification raised when a sensor reading requires attention.

    Alerts are ordered by severity so that they can be placed in a
    priority queue (e.g. ``heapq``) and processed with the most severe
    alerts surfaced first.

    Attributes:
        sensor_id: Identifier of the sensor that triggered the alert.
        severity: One of "LOW", "MEDIUM", "HIGH", "CRITICAL".
        message: Human-readable description of the alert condition.
        timestamp: When the alert was generated.
    """

    _SEVERITY_RANK = {
        "LOW": 0,
        "MEDIUM": 1,
        "HIGH": 2,
        "CRITICAL": 3,
    }

    def __init__(
        self,
        sensor_id: str,
        severity: str,
        message: str,
        timestamp: datetime,
```

```
) -> None:
    """Initialize an Alert.

    Args:
        sensor_id: Identifier of the sensor that triggered the alert.
        severity: Severity level. Must be one of "LOW", "MEDIUM",
            "HIGH", or "CRITICAL".
        message: Human-readable description of the alert condition.
        timestamp: When the alert was generated.

    Raises:
        ValueError: If ``severity`` is not one of the allowed values.
    """
    if severity not in self._SEVERITY_RANK:
        allowed = ", ".join(self._SEVERITY_RANK)
        raise ValueError(
            f"Invalid severity {severity!r}; must be one of: {allowed}"
        )

    self.sensor_id = sensor_id
    self.severity = severity
    self.message = message
    self.timestamp = timestamp

def __lt__(self, other: "Alert") -> bool:
    """Compare alerts by severity rank for priority-queue ordering.

    An alert is considered "less than" another if its severity is
    lower (e.g. LOW < MEDIUM < HIGH < CRITICAL).

    Args:
        other: The other Alert to compare against.

    Returns:
        True if this alert's severity rank is lower than ``other``'s.
    """
    if not isinstance(other, Alert):
        return NotImplemented
    return self._SEVERITY_RANK[self.severity] < self._SEVERITY_RANK[other.sev

def __str__(self) -> str:
    """Return a human-readable summary of the alert."""
    return f"[{self.severity}] {self.sensor_id}: {self.message}"

def __repr__(self) -> str:
    """Return an unambiguous, eval-able representation of the alert."""
    return (
```

```
            f"Alert(sensor_id={self.sensor_id!r}, severity={self.severity!r}, "
            f"message={self.message!r}, timestamp={self.timestamp!r})"
        )


class AlertRule:
    """A threshold-based rule that evaluates readings from a single sensor.

    A rule defines an acceptable value range ``[min_threshold,
    max_threshold]`` for one sensor, identified by ``sensor_id``. Readings
    from other sensors are ignored. Readings outside the range produce an
    :class:`Alert`, with severity scaled by how far outside the range the
    value falls, relative to the threshold that was breached:

        * within 10% over/under the breached threshold -> "MEDIUM"
        * more than 10% but up to 25% -> "HIGH"
        * more than 25% -> "CRITICAL"

    Attributes:
        sensor_id: Identifier of the sensor this rule applies to.
        min_threshold: Minimum acceptable value (inclusive).
        max_threshold: Maximum acceptable value (inclusive).
    """

    _MEDIUM_LIMIT = 0.10
    _HIGH_LIMIT = 0.25

    def __init__(
        self,
        sensor_id: str,
        min_threshold: float,
        max_threshold: float,
    ) -> None:
        """Initialize an AlertRule.

        Args:
            sensor_id: Identifier of the sensor this rule applies to.
            min_threshold: Minimum acceptable value (inclusive).
            max_threshold: Maximum acceptable value (inclusive).
        """
        self.sensor_id = sensor_id
        self.min_threshold = min_threshold
        self.max_threshold = max_threshold

    def check(self, reading: SensorReading) -> Optional[Alert]:
        """Evaluate a reading against this rule.
```

```
    Args:
        reading: The sensor reading to evaluate.

    Returns:
        An Alert if ``reading.sensor_id`` matches this rule's
        ``sensor_id`` and ``reading.value`` falls outside
        ``[min_threshold, max_threshold]``; otherwise None.
    """
    if reading.sensor_id != self.sensor_id:
        return None

    if self.min_threshold <= reading.value <= self.max_threshold:
        return None

    if reading.value > self.max_threshold:
        breached_threshold = self.max_threshold
        direction = "above maximum"
    else:
        breached_threshold = self.min_threshold
        direction = "below minimum"

    deviation_fraction = self._deviation_fraction(
        reading.value, breached_threshold
    )
    severity = self._severity_for_deviation(deviation_fraction)

    message = (
        f"Reading {reading.value}{reading.unit} is {direction} "
        f"threshold {breached_threshold}{reading.unit} "
        f"({deviation_fraction:.1%} deviation)"
    )

    return Alert(
        sensor_id=reading.sensor_id,
        severity=severity,
        message=message,
        timestamp=reading.timestamp,
    )

@staticmethod
def _deviation_fraction(value: float, breached_threshold: float) -> float:
    """Compute how far ``value`` deviates from a breached threshold.

    Args:
        value: The out-of-range reading value.
        breached_threshold: The threshold that was breached.
```

```
    Returns:
        The absolute deviation as a fraction of the threshold's
        magnitude. If the threshold is zero, returns infinity for any
        nonzero deviation (any breach is treated as maximally severe),
        and zero otherwise.
    """
    if breached_threshold == 0:
        return float("inf") if value != breached_threshold else 0.0
    return abs(value - breached_threshold) / abs(breached_threshold)

@classmethod
def _severity_for_deviation(cls, deviation_fraction: float) -> str:
    """Map a deviation fraction to a severity level.

    Args:
        deviation_fraction: Absolute deviation from the breached
            threshold, as a fraction of the threshold's magnitude.

    Returns:
        "MEDIUM" if within 10%, "HIGH" if within 25%, otherwise
        "CRITICAL".
    """
    if deviation_fraction <= cls._MEDIUM_LIMIT:
        return "MEDIUM"
    if deviation_fraction <= cls._HIGH_LIMIT:
        return "HIGH"
    return "CRITICAL"

def __str__(self) -> str:
    """Return a human-readable summary of the rule."""
    return (
        f"AlertRule for {self.sensor_id}: "
        f"[{self.min_threshold}, {self.max_threshold}]"
    )

def __repr__(self) -> str:
    """Return an unambiguous, eval-able representation of the rule."""
    return (
        f"AlertRule(sensor_id={self.sensor_id!r}, "
        f"min_threshold={self.min_threshold!r}, "
        f"max_threshold={self.max_threshold!r})"
    )
```